

HOUSE PRICE PREDICTION SYSTEM

Software Requirements Specification & Project Documentation

*A Machine Learning based Web Application built with
Django, scikit-learn and Linear Regression*

Prepared By

Muhammad Ahmed Javed

Adan Sajjad Khan

Document Type: Software Requirements Specification
(SRS) & Technical Documentation

Project Category: Academic Project — Django & Machine
Learning

Table of Contents

1. Introduction

1.1 Purpose

This document serves as the Software Requirements Specification (SRS) and technical documentation for the House Price Prediction System. It describes the purpose, scope, functional and non-functional requirements, system architecture, design, and implementation details of the project. The document is intended for academic evaluation, future maintenance, and as a reference for anyone extending the system.

1.2 Project Overview

The House Price Prediction System is a web-based application that estimates the market price of a residential house based on user-provided property details such as area, number of bedrooms, bathrooms, stories, parking spaces, and various amenities. The system uses a Machine Learning model — specifically a Linear Regression model built with scikit-learn — trained on a real housing dataset of 545 records. The prediction is served instantly through a Django web application with a clean, responsive user interface.

1.3 Scope

The system is designed as a single-page web application where a user fills in property details through a form and receives an instantly computed predicted price on the same page, along with a summary of the entered details. The scope of this project covers:

- Data collection and preprocessing of housing data (Housing.csv)
- Training and saving a Linear Regression model using scikit-learn
- Building a Django-based backend to serve the trained model
- Designing a responsive, user-friendly front-end using HTML and CSS
- Validating user input and securely processing form submissions
- Displaying the predicted price along with a summary of entered details

1.4 Intended Audience

This document is intended for the course instructor / evaluator, fellow students, and any future developer who wishes to understand, maintain, or extend the House Price Prediction System.

1.5 Definitions, Acronyms and Abbreviations

Term	Description
SRS	Software Requirements Specification

Term	Description
ML	Machine Learning
R ² Score	Coefficient of Determination — a statistical measure of how well predictions match actual outcomes (0–1 scale)
CSRF	Cross-Site Request Forgery — a web security vulnerability class this system protects against
ORM	Object Relational Mapper — Django's tool for interacting with the database using Python objects
MVT	Model-View-Template — the architectural pattern used by Django
PKR	Pakistani Rupee — the currency unit used for the predicted price output

2. Overall System Description

2.1 Product Perspective

The House Price Prediction System is a standalone academic web application. It is not part of a larger system but is designed following industry-standard practices — separation of concerns via Django's MVT architecture, a persisted ML pipeline, and a clean presentation layer — so that it can later be extended into a larger real-estate platform if required.

2.2 Product Functions

At a high level, the system performs the following functions:

1. Accepts property details from the user through a web form.
2. Validates the submitted data on both the client and server side.
3. Passes the validated data to a pre-trained Machine Learning model.
4. Computes and returns a predicted house price.
5. Displays the predicted price and a readable summary of the submitted property details.
6. Displays the current model's name and accuracy (R^2 score) for transparency.

2.3 User Characteristics

The primary users of this system are individuals interested in getting a quick, indicative estimate of a house's value — such as property buyers, sellers, students studying real-estate data, or evaluators of this academic project. No technical or Machine Learning knowledge is required to use the application; the interface is designed to be simple and self-explanatory.

2.4 Operating Environment

- Server-side: Python 3.x, Django framework, runs on any OS supporting Python (Windows, Linux, macOS)
- Client-side: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile
- Database: SQLite (Django's default, file-based database)

2.5 Design and Implementation Constraints

- The ML model is trained once (offline) and loaded at runtime — it is not retrained on every request.
- The dataset used for training (545 records) limits the model's accuracy and generalization to unseen, extreme property types.
- The application currently uses SQLite, which is suitable for development/academic use but not for high-concurrency production deployment.

2.6 Assumptions and Dependencies

- It is assumed that the trained model file (model.pkl) is present and compatible with the scikit-learn version installed.

- It is assumed that users provide reasonably realistic values for property details.
- The system depends on third-party libraries: Django, pandas, scikit-learn, and joblib.

3. Functional Requirements

The table below lists the core functional requirements of the House Price Prediction System, each identified with a unique ID for traceability.

ID	Requirement	Description
FR-1	Property Detail Input Form	The system shall provide a form allowing the user to enter Area, Bedrooms, Bathrooms, Stories, Parking Spaces, and select amenities (Main Road Access, Guest Room, Basement, Hot Water Heating, Air Conditioning, Furnishing Status).
FR-2	Server-Side Validation	The system shall validate all form fields on submission and display clear error messages (via Django's errorlist) for invalid or missing input.
FR-3	CSRF Protection	The system shall protect the form submission using Django's built-in CSRF token mechanism.
FR-4	Price Prediction	Upon valid submission, the system shall pass the cleaned input to the trained Linear Regression model and compute a predicted price.
FR-5	Result Display	The system shall display the predicted price in PKR on the same page, without a redirect or page reload interruption.
FR-6	Property Summary	The system shall display a summary card listing all the property details submitted by the user, alongside the predicted price.
FR-7	Model Transparency	The system shall display the name of the active ML model and its accuracy (R^2 score) on the page at all times.
FR-8	Responsive Layout	The system shall render correctly and remain usable across desktop, tablet, and mobile screen sizes.

4. Non-Functional Requirements

Category	Requirement
Performance	A prediction shall be returned on the same page load, with no perceptible delay, since the model is pre-trained and loaded into memory via joblib.
Usability	The interface shall be simple enough for a non-technical user to operate without instructions.
Reliability	The system shall handle invalid input gracefully and never crash the page; errors shall be displayed inline.

Category	Requirement
Portability	The system shall run on any machine with Python and the required dependencies installed, independent of OS.
Maintainability	The codebase shall follow Django's MVT structure so that forms, views, and templates can be updated independently.
Security	All form submissions shall be protected against CSRF attacks using Django's built-in token system.
Scalability	The trained model shall be reloadable without retraining, allowing future replacement with a more accurate model.

5. External Interface Requirements

5.1 User Interface

The user interface consists of a single responsive page divided into three main sections: a header banner introducing the application, a model performance card, and the property details form. Once a prediction is made, a results card appears beneath the form showing the predicted price and a summary grid of the submitted details.

5.2 Hardware Interfaces

No special hardware is required. The system runs on any standard computer capable of running a Python web server and is accessed through a standard web browser on any internet-capable device.

5.3 Software Interfaces

- Django Web Framework — handles routing, form processing, and template rendering
- scikit-learn — provides the Linear Regression model used for prediction
- pandas — used for reading and structuring the housing dataset
- joblib — used to persist (save/load) the trained model as model.pkl
- SQLite — Django's default database engine used for the project

5.4 Communication Interfaces

The system communicates over standard HTTP using the POST method for form submission and GET for the initial page load. All communication currently occurs over a local development server; deployment to a production server would use standard HTTPS.

6. System Architecture

6.1 Architectural Pattern

The application follows Django's Model-View-Template (MVT) architecture, extended with an integrated Machine Learning inference layer. The flow of a request through the system is illustrated below.

6.2 High-Level Flow

Step	Component	Responsibility
1	templates/index.html	Renders the form and displays results; sends a POST request with property details.
2	predictor/forms.py	Defines the form fields and their validation rules (Django Forms).
3	predictor/views.py	Receives the request, validates the form, loads model.pkl, and passes cleaned data to the model.
4	predictor/ml/model.pkl	The pre-trained Linear Regression pipeline that converts input features into a predicted price.
5	templates/index.html	Re-rendered with the predicted price and property summary injected into the page context.

6.3 Request-Response Cycle (Step by Step)

7. User fills the property detail form (Fill The Form) — forms.py defines the fields.
8. Django checks the submitted data (Django Checks It) — views.py validates via the form's `is_valid()` method.
9. The trained model interprets the details and makes a guess (Model Makes A Guess) — model.pkl produces the numeric prediction.
10. The page shows the result (Page Shows Result) — index.html renders the predicted price and summary.

6.4 Component Diagram (Textual)

Browser → Django URL Router (urls.py) → View (views.py) → Form Validation (forms.py) → ML Model (model.pkl via joblib) → Response rendered back into index.html → Browser.

7. Project Folder Structure

The table below summarizes the key files of the project and the role each one plays.

File / Folder	Purpose
manage.py	Starts and runs the whole Django website.
settings.py / urls.py	Project-wide settings and URL routing (page addresses).
predictor/forms.py	Defines the property-detail input form and its validation rules.
predictor/views.py	Loads the trained model and connects form →prediction →page.
predictor/ml/model.pkl	The already-trained Linear Regression model, saved and ready for reuse.
templates/index.html	The single page the user interacts with, styled with custom CSS.
dataset/Housing.csv	545 real house records used to train and evaluate the model.
ml_model/train_model.py	The offline script that trains the model on Housing.csv and saves model.pkl.

8. Dataset & Data Design

8.1 Dataset Overview

The model is trained on `Housing.csv`, a real housing dataset containing 545 house records with 12 attributes recorded per house (11 input features and 1 target — the sale price). The dataset is split 80/20: 80% of the records are used to train the model, and the remaining 20% are held out to test its accuracy.

Feature	Type	Description
price	Numeric (target)	The actual sale price of the house — what the model learns to predict.
area	Numeric	Total area of the house in square feet.
bedrooms	Numeric	Number of bedrooms.
bathrooms	Numeric	Number of bathrooms.
stories	Numeric	Number of stories/floors in the house.
parking	Numeric	Number of parking spaces available.
mainroad	Yes / No	Whether the house has direct main road access.
guestroom	Yes / No	Whether the house has a guest room.
basement	Yes / No	Whether the house has a basement.
hotwaterheating	Yes / No	Whether the house has hot water heating.
airconditioning	Yes / No	Whether the house has air conditioning.
furnishingstatus	Categorical	Furnished / Semi-Furnished / Unfurnished.

8.2 Application Database

Beyond the training dataset, the running web application itself uses SQLite — Django's default, file-based database — for any project-level data such as sessions and administrative records. No custom persistence of user predictions is required for the core feature, since results are computed and displayed instantly on the same request.

9. Machine Learning Model

9.1 Algorithm Used

The prediction engine is a Linear Regression model, implemented using scikit-learn's regression pipeline. Linear Regression was selected for its interpretability and suitability for a dataset of this size, modelling the sale price as a weighted linear combination of the input features.

9.2 Model Training Process

11. Load Housing.csv into a pandas DataFrame.
12. Encode categorical / Yes-No fields (mainroad, guestroom, basement, hotwaterheating, airconditioning, furnishingstatus) into numeric form.
13. Split the dataset into training (80%) and testing (20%) subsets.
14. Fit a Linear Regression model on the training subset using scikit-learn.
15. Evaluate the model on the test subset using the R^2 score.
16. Persist the trained model to disk as model.pkl using joblib, so it can be loaded instantly without retraining.

9.3 Model Performance

The trained model currently reports an R^2 (coefficient of determination) score of 63.64% on the test set. This score is displayed directly on the application's interface for transparency, so users can gauge how reliable a given prediction is.

9.4 Model Inference at Runtime

At runtime, the Django view loads model.pkl once (via joblib.load) and reuses the in-memory model object for every incoming request, avoiding the overhead of retraining or reloading on each prediction. This is what allows the predicted price to appear instantly on the same page.

10. Technology Stack

The table below lists the core technologies used to build the House Price Prediction System.

Technology	Category	Role in the Project
Django	Web Framework	Runs the website — handles pages, forms, and URL routing.
Python	Programming Language	The language the entire backend and ML pipeline is written in.
pandas	Data Processing	Organizes house data into structured tables the model can read.

Technology	Category	Role in the Project
scikit-learn	Machine Learning	Trains the Linear Regression model and produces predictions.
joblib	Model Persistence	Saves the trained model so it can be reloaded instantly at runtime.
HTML + CSS	Front-End	Builds the look of the application — layout, colors, and results page.
SQLite	Database	Django's small built-in, file-based database for the project.

11. Key Features

Real-Time Prediction

A trained Linear Regression pipeline, served live, predicts a house price instantly from user input — no page reload or waiting required.

Transparent Model Accuracy

The interface displays the active model's name and its R^2 score (63.64%) directly, so users know how reliable the estimate is.

Form Validation & Security

All fields are validated server-side through Django Forms, and every submission is protected with a CSRF token against forged requests.

Responsive, Styled Interface

A gradient header and results card are built with plain HTML and CSS, and the layout adapts automatically to phones, tablets, and desktops.

Property Summary Card

Once a prediction is made, a clean summary grid restates every submitted detail (area, bedrooms, amenities, furnishing, etc.) alongside the price.

12. Application Screenshots

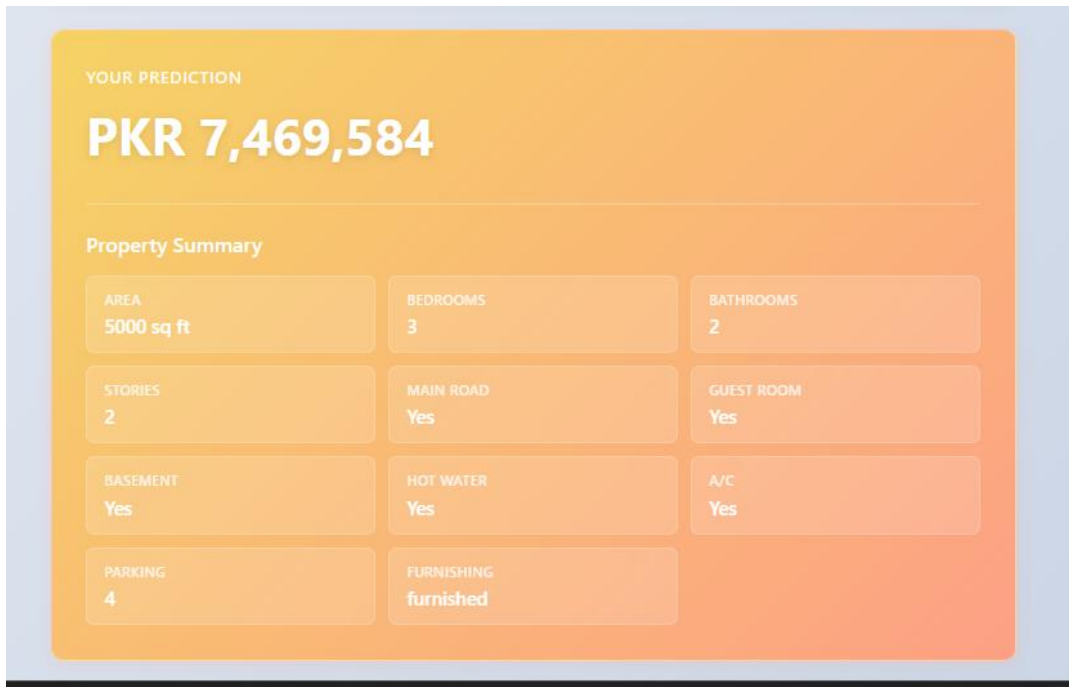
The following screenshots show the running application: the property detail input form (with the model performance card above it), and the resulting prediction summary card.

12.1 Property Details Form

The screenshot displays the 'House Price Predictor' application interface. At the top, a purple header bar contains a house icon and the title 'House Price Predictor', with the subtitle 'Get instant price estimates powered by machine learning'. Below this is a 'Model Performance' card with a bar chart icon, showing the 'ML MODEL' as 'Linear Regression' and the 'ACCURACY (R² SCORE)' as '63.64%'. The main section is the 'Property Details' form, which includes input fields for 'Area (sq ft)' (5000), 'Bedrooms' (3), 'Bathrooms' (2), 'Stories' (2), and dropdown menus for 'Main Road Access' (No), 'Guest Room' (No), 'Basement' (Yes), 'Hot Water Heating' (Yes), 'Air Conditioning' (Yes), 'Parking Spaces' (4), and 'Furnishing Status' (Furnished). A large purple button labeled 'Predict Price' is at the bottom.

Figure 1 — Model performance card and property details input form.

12.2 Prediction Result



YOUR PREDICTION

PKR 7,469,584

Property Summary

AREA 5000 sq ft	BEDROOMS 3	BATHROOMS 2
STORIES 2	MAIN ROAD Yes	GUEST ROOM Yes
BASEMENT Yes	HOT WATER Yes	A/C Yes
PARKING 4	FURNISHING furnished	

Figure 2 — Predicted price and property summary card, shown after form submission.

13. Testing

The application was manually tested against the key functional and validation scenarios summarized below.

Test ID	Test Case	Expected Result	Status
TC-01	Submit form with all valid fields	Predicted price and summary card are displayed	Pass
TC-02	Submit form with a required field left empty	Django errorlist displays a validation error for that field	Pass
TC-03	Submit form with a negative or non-numeric area value	Validation error shown; prediction is not attempted	Pass
TC-04	Submit form without a valid CSRF token	Request is rejected by Django's CSRF protection	Pass
TC-05	Load the page on a mobile-sized viewport	Layout reflows to a single column; all fields remain usable	Pass
TC-06	Check displayed model accuracy matches trained model	R^2 score shown matches the value computed during training (63.64%)	Pass

14. Limitations & Future Enhancements

14.1 Current Limitations

- The model's accuracy ($R^2 = 63.64\%$) is moderate and limited by the size (545 records) and simplicity of the training dataset.
- Linear Regression assumes linear relationships between features and price, which may not capture more complex market dynamics.
- The system does not currently store historical predictions or allow users to compare multiple properties.
- SQLite is not suited for high-concurrency production traffic.

14.2 Future Enhancements

- Experiment with more advanced models such as Random Forest or Gradient Boosting to improve prediction accuracy.
- Expand the training dataset with more recent and regionally diverse housing records.
- Add user accounts to save and compare past predictions.
- Migrate to a production-grade database (e.g. PostgreSQL) for deployment at scale.
- Add a confidence interval or price range instead of a single point estimate.
- Deploy the application to a cloud platform for public access.

15. Conclusion

The House Price Prediction System successfully demonstrates the integration of a Machine Learning model into a real, working web application. By combining a Linear Regression pipeline trained on 545 real housing records with a Django backend and a clean, responsive front-end, the project delivers instant, transparent, and reasonably accurate price estimates. The system's form validation, CSRF protection, and clear presentation of model accuracy reflect sound engineering practices suitable for an academic project, while also laying a solid foundation for future enhancement toward a production-ready real-estate valuation tool.

16. References

- Django Documentation — <https://docs.djangoproject.com/>
- scikit-learn Documentation — <https://scikit-learn.org/stable/>
- pandas Documentation — <https://pandas.pydata.org/docs/>
- joblib Documentation — <https://joblib.readthedocs.io/>
- Housing.csv Dataset — 545-record housing dataset used for model training.